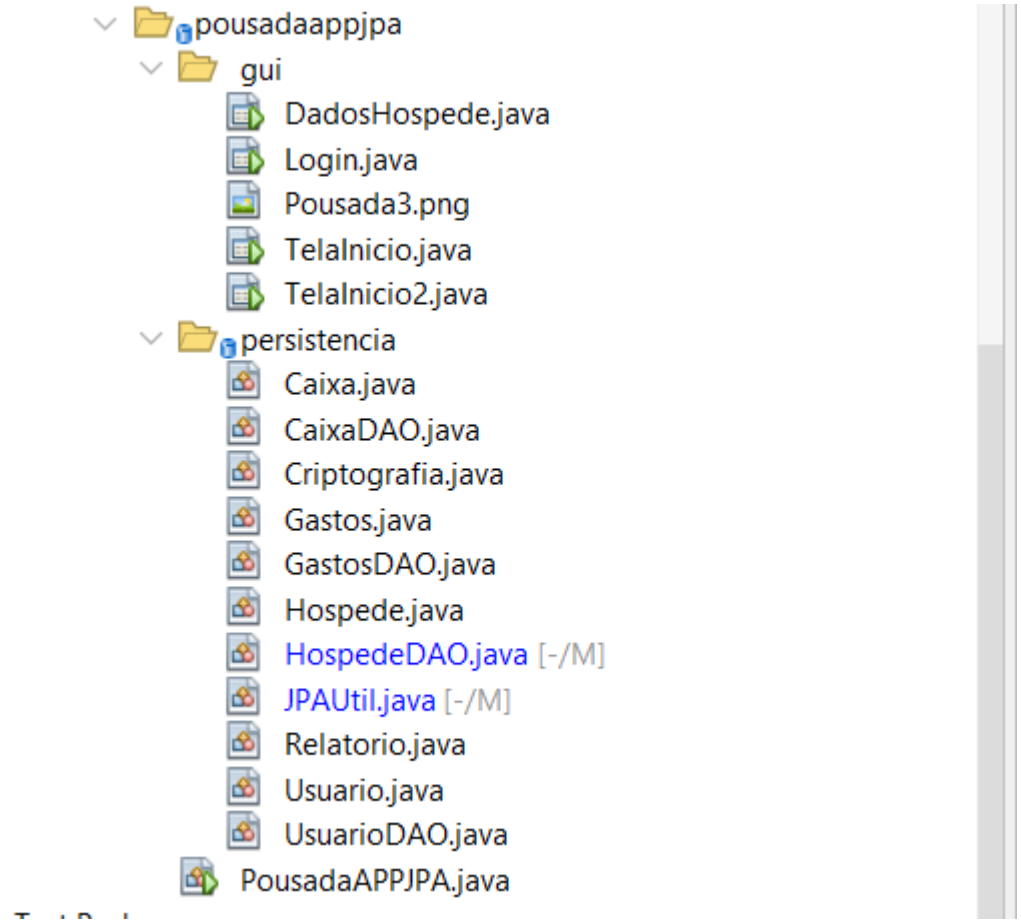
# Relatório de Refatoração – Projeto Integrador (PI) Modulo B

## Aluno: Filipe Luiz Couto

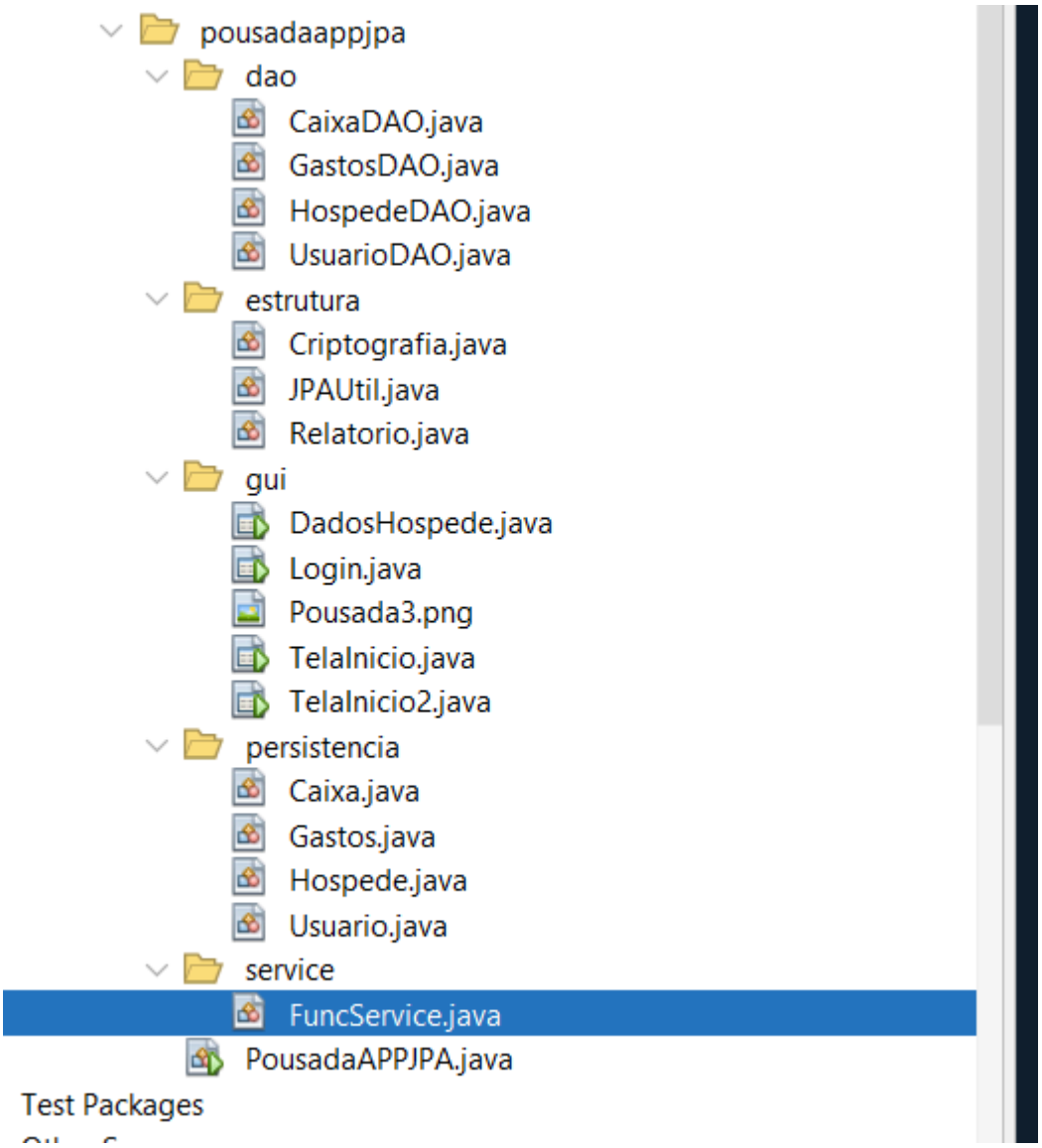
## PousadaAppJPA

## Estrutura Anterior




Na versão inicial, o projeto estava organizado em dois pacotes principais:
● gui: continha classes de interface gráfica (Swing), mas também acumulava lógica de negócio em alguns pontos.
● persistencia: reunia tanto entidades quanto DAOs e utilitários, sem clara separação de responsabilidades.
Problema identificado: mistura de responsabilidades (interface + regras de negócio + persistência), dificultando reutilização no sistema web.

## Estrutura Atual




Na versão refatorada, o projeto foi reorganizado em pacotes distintos:
● dao: classes responsáveis pelo acesso a dados (CaixaDAO, GastosDAO, HospedeDAO, UsuarioDAO).
● persistencia: apenas entidades (Caixa, Gastos, Hospede, Usuario).
● estrutura: utilitários e classes de suporte (Criptografia, JPAUtil, Relatorio).
● gui: interface gráfica, isolada da lógica.
● service: camada de serviços, criada para centralizar regras de negócio e orquestrar chamadas aos DAOs.
Destaque: criação da classe FuncService, responsável por encapsular operações de negócio (cadastro, listagem, exclusão, autenticação, movimentação de caixa etc.), aplicando validações e regras antes de acessar os DAOs.

## Princípios SOLID Aplicados

● SRP (Responsabilidade Única):
○ Antes: classes misturavam interface e lógica de negócio.
○ Depois: cada camada tem sua função clara (GUI só interface, DAO só persistência, Service só regras de negócio).
○ Exemplo: FuncService concentra regras de negócio, enquanto DAOs cuidam apenas de persistência.
● OCP (Aberto/Fechado):
○ DAOs e serviços podem ser estendidos sem alterar código existente.
○ Exemplo: novos métodos de relatório podem ser adicionados em Relatorio sem modificar classes já estáveis.
● DIP (Inversão de Dependência):
○ A lógica de negócio depende de abstrações (interfaces DAO), não de implementações concretas.
○ Isso facilita testes e substituição de camadas.

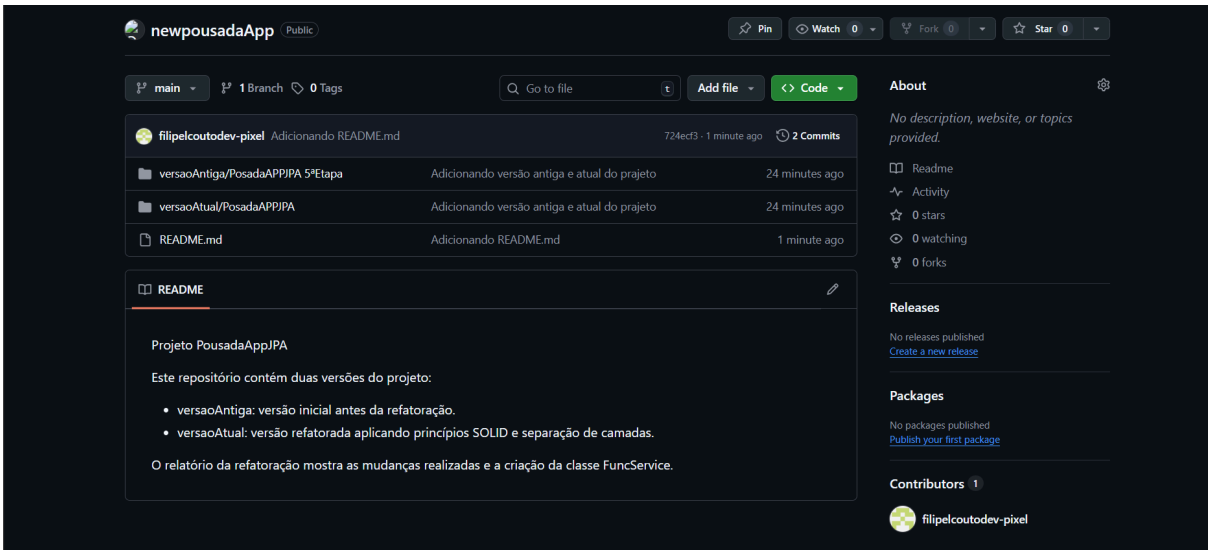
## Refatorações Realizadas

● Separação de pacotes: entidades, DAOs, serviços e interface gráfica.
● Criação da classe FuncService: centraliza regras de negócio e validações, evitando que a GUI acesse diretamente os DAOs.
● Validações adicionadas: exemplo em cadastrarUsuario() e autenticar(), garantindo consistência antes de persistir dados.
● Métodos longos simplificados: extração de responsabilidades em métodos menores e mais claros.
● Aplicação de padrão MVC:
○ Model: entidades em persistencia.
○ View: classes em gui.
○ Controller/Service: FuncService e DAOs.

## Evidências

Para comprovar a refatoração e a correta organização do projeto, o código foi versionado e publicado em um repositório público no GitHub. Nele estão disponíveis as duas versões do sistema:
● versaoAntiga: versão inicial antes da refatoração.
● versaoAtual: versão refatorada aplicando princípios SOLID e separação de camadas.
Além disso, foi criado um arquivo README.md explicativo, descrevendo as diferenças entre as versões e destacando a criação da classe FuncService.

A imagem abaixo mostra a estrutura do repositório:




O projeto pode ser acessado diretamente pelo link:
https://github.com/filipecoutodev-pixel/newpousadaApp

## Testes realizados no método `main()`

Para validar o funcionamento do sistema após a refatoração, foram implementados testes simples no método `main()`. Esses testes consistem em criar registros de **Hospede** e **Gastos**, e verificar se o **Caixa** é atualizado corretamente.

```
// Teste de Hospede
    Hospede hospede = new Hospede();
    hospede.setNome("Pedro");
    hospede.setCpf("000.000.000-01");
    hospede.setIdade(23);
    hospede.setContato("24-999784563");
    hospede.setNumeroDeDias(3);
    hospede.setIdentificacaoQuarto("Quarto 1");
    hospede.setSaldo(360.00);

    HospedeDAO hospedeDAO = new HospedeDAO();
    hospedeDAO.cadastrarHospede(hospede);

    System.out.println("Hospede salvo: " + hospede.getNome());

    // Teste de Gastos
    Gastos gasto = new Gastos();
    gasto.setData("30/03/2026");
    gasto.setDescricao("Pães");
```

```
gasto.setValor(120.00);

GastosDAO gastosDAO = new GastosDAO();
gastosDAO.cadastrarGasto(gasto);

System.out.println("Gasto salvo: " + gasto.getDescricao());

Login login = new Login();
login.setVisible(true);
```

## Observação

Para que o sistema funcione corretamente, é necessário criar pelo menos um usuário com tipo de **Administrador** e outro com tipo **Secretaria** por exemplo. Para garantir acesso às funcionalidades de login e controle de permissões.